

Representación de la información

Departamento de Enseñaría de Computadores

Representación de la información

- 1 Definiciones básicas
- 2 Transformación entre bases
- 3 Codificación de números enteros con signo
- 4 Codificación de números reales
- 5 Codificación de caracteres

Definiciones básicas

- N : número válido en el sistema de numeración.
- b : base del sistema de numeración. Número de símbolos permitidos en el sistema.
- d_i : un símbolo cualquiera de los permitidos en el sistema de numeración.
- n : número de dígitos de la parte entera.
- k : número de dígitos de la parte fraccionaria.
- $.$: separador de la parte entera y la fraccionaria.

Valor en el sistema decimal de una cantidad expresada en ese sistema de numeración:

$$\begin{aligned} N = \langle d_{n-1} \dots d_1 d_0 . d_{-1} \dots d_{-k} \rangle &= \sum_{i=-k}^{n-1} d_i b^i \\ &= d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{aligned}$$

Ejemplos

- **Binario a decimal:** $b=2$, $S=0,1$.

$$\begin{aligned}1101.01_2 &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} \\ &= 8 + 4 + 1 + 0.25 = 13.25\end{aligned}$$

- **Base 5 a decimal:** $b=5$, $S=0,1,2,3,4$.

$$\begin{aligned}130.14_5 &= 1 \cdot 5^2 + 3 \cdot 5^1 + 0 \cdot 5^0 + 1 \cdot 5^{-1} + 4 \cdot 5^{-2} \\ &= 25 + 15 + 0.2 + 0.16 = 40.36\end{aligned}$$

- **Hexadecimal a decimal:** $b=16$,
 $S=0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F$ donde
 $A=10, B=11, C=12, D=13, E=14$ y $F=15$.

$$3e.b = 3 \cdot 16^1 + 14 \cdot 16^0 + 11 \cdot 16^{-1} = 48 + 14 + 0.6875 = 62.6875$$

Transformación de un número decimal a base b

Para hacer el cambio inverso se deben convertir la parte decimal y la entera por separado.

- La parte entera se va dividiendo por la base hasta que el cociente sea cero. La parte entera del número en base b son los restos de las divisiones sucesivas, empezando por la última.
- La parte decimal se va multiplicando por b hasta llegar a un número entero o hasta alcanzar el número de cifras decimales deseadas. La parte entera se pone a 0 en cada paso. La parte decimal son las partes enteras de las multiplicaciones.
- Ejemplos:
 - $25.375_{10} = 11001.011_2$
 - $1025_{10} = 10000000001_2$
 - $37.1_{10} = \overline{100101.0001100110011001100110011001100110011...}_2 = 100101.000\overline{11}_2$

Transformación entre bases

Bases distintas a 10

Para convertir entre dos bases distintas a 10: convertimos a base 10 y luego pasamos el resultado a la base final. Para la equivalencia podemos hacer una tabla:

Base 10	0	1	2	3	4	5	6	7
Base 16	0	1	2	3	4	5	6	7
Base 2	0000	0001	0010	0011	0100	0101	0110	0111

Base 10	8	9	10	11	12	13	14	15
Base 16	8	9	a	b	c	d	e	f
Base 2	1000	1001	1010	1011	1100	1101	1110	1111

Ejemplo:

$$a2.b_{16} = 10100010.1011_2$$

Transformación entre bases

Bases distintas a 10

Si queremos pasar un número a una base mayor tenemos que agrupar. Si es necesario añadimos ceros NO SIGNIFICATIVOS (a la izquierda para la parte entera, a la derecha para la parte decimal):

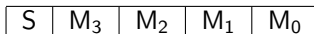
Ejemplo: Convertir 1001100_2 a base 16.

$$100\ 1100_2 = 0100\ 1100_2 = \underbrace{0100}_4\ \underbrace{1100}_c = 4c_{16} = 0x4c$$

Codificación de números enteros con signo

Signo-magnitud

- En **signo-magnitud** los valores binarios se representan como sigue:



S: bit de signo (0: positivo, 1: negativo); M₃ ... M₀: magnitud.

- Rango de valores con n bits: $[-2^{n-1} + 1, 2^{n-1} - 1]$.
- Ejemplo:** Números posibles en signo-magnitud con 4 bits.

Valor	0	1	2	3	4	5	6	7
Binario	0000	0001	0010	0011	0100	0101	0110	0111

Valor	-0	-1	-2	-3	-4	-5	-6	-7
Binario	1000	1001	1010	1011	1100	1101	1110	1111

Hay 2 representaciones para el 0.

Codificación de números enteros con signo

Complemento a 1

- En **complemento a 1**, los valores se representan como sigue:
 - Números **positivos**: igual que en signo–magnitud
 - Números **negativos**: se invierten todos los bits del número positivo.
- Podemos representar el mismo número de valores que en signo–magnitud.
- Al igual que ocurre en signo–magnitud, los positivos empiezan por 0 y los negativos por 1.
- Los *overflows* (desbordamiento del formato) se detectan fácilmente: si al sumar dos números positivos nos da como resultado un número negativo (o viceversa), ha habido *overflow*. Nunca habrá *overflow* al sumar un número negativo con uno positivo.

Codificación de números enteros con signo

Complemento a 1

Ejemplo: Para 3 bits.

Valor	-3	-2	-1	-0	0	1	2	3
Binario	100	101	110	111	000	001	010	011

Operaciones:

$$1 + 2 = 001 + 010 = 011 = 3$$

$$3 - 2 = 3 + (-2) = 011 + 101 = 1000 \text{ (acarreo)}$$

$$= 000 + 1 = 001 = 1$$

$$3 + 3 = 011 + 011 = 110 = -1$$

(el resultado no es correcto por *overflow*)

Codificación de números enteros con signo

Complemento a 1

Ejemplo: Para 4 bits.

Podemos representar positivos hasta $2^{4-1} - 1 = 7$.

Valor	0	1	2	3	4	5	6	7
Binario	0000	0001	0010	0011	0100	0101	0110	0111

Para los números negativos haríamos lo mismo, pero invirtiendo los bits:

Valor	-0	-1	-2	-3	-4	-5	-6	-7
Binario	1111	1110	1101	1100	1011	1010	1001	1000

Operaciones:

$$3 + 2 = 0011 + 0010 = 0101 = 5$$

$$3 - 2 = 3 + (-2) = 0011 + 1101 = 10000 \text{ (acarreo)}$$

$$\rightarrow 0000 + 1 = 0001 = 1$$

$$3 - 5 = 3 + (-5) = 0011 + 1010 = 1101 = -2$$

$$-3 - 2 = -3 + (-2) = 1100 + 1101 = 11001 \text{ (acarreo)}$$

$$\rightarrow 1001 + 1 = 1010 = -5$$

Codificación de números enteros con signo

Complemento a 2

- En **Complemento a 2**, los valores se representan como sigue:
 - Números **positivos**: igual que en Complemento a 1
 - Números **negativos**: igual que en Complemento a 1 sumando 1
- Rango de valores con n bits: $[-2^{n-1}, 2^{n-1} - 1]$.
- Se opera de forma similar a complemento a 1, pero el acarreo se descarta.
- Los *overflows* se detectan igual que en complemento a 1.
- Ventajas:
 - El cero tiene una única representación
 - La suma es más simple que en complemento a 1

Codificación de números enteros con signo

Complemento a 2

Ejemplo: Para 4 bits.

Valor	0	1	2	3	4	5	6	7
Binario	0000	0001	0010	0011	0100	0101	0110	0111

Valor	-0	-1	-2	-3	-4	-5	-6	-7	-8
Binario	0000	1111	1110	1101	1100	1011	1010	1001	1000

$$3 + 2 = 0011 + 0010 = 0101 = 5$$

$$3 - 2 = 3 + (-2) = 0011 + 1110 = 10001(\text{acarreo}) \rightarrow 0001 = 1$$

$$3 - 5 = 3 + (-5) = 0011 + 1011 = 1110 = -2$$

$$-3 - 2 = (-3) + (-2) = 1101 + 1110 = 11011(\text{acarreo}) \rightarrow 1011 = -5$$

Codificación de números enteros con signo

Exceso a $2^{n-1} - 1$

- Se suma $(2^{n-1} - 1)$ antes de pasarlo a binario.
- Ejemplo:** Representación de los números de 4 bits en exceso a 7:

Valor	-7	-6	-5	-4	-3	-2	-1	0
Binario	0000	0001	0010	0011	0100	0101	0110	0111

Valor	1	2	3	4	5	6	7	8
Binario	1000	1001	1010	1011	1100	1101	1110	1111

- Es complicado operar con números en "exceso a", pero tiene la ventaja de seguir el mismo orden de menor a mayor en binario y en decimal.

Codificación de números reales

Representación en coma fija y flotante

Representación en coma fija:

- El número de bits de la parte entera y fraccionaria es fijo
- Solo unas pocas fracciones pueden ser representadas → problemas de precisión

Representación en coma flotante:

- Premisa: cualquier número real N en base b puede ser expresado como $N = m \times b^e$
- **Ejemplos:** Números en notación científica
$$823.54 = 8.2354 \cdot 10^2$$
$$0.00653 = 6.53 \cdot 10^{-3}$$
$$100101.1001_2 = 1.001011001_2 \cdot 2^5$$
- Para representar un número real binario solo necesitamos almacenar la mantisa y el exponente, la base se da por conocida

Codificación de números reales

IEEE754

- El estandar IEEE 754 es el más utilizado para representación de números reales en ordenadores.
- Define precisión simple (32 bits) y doble (64 bits).
- En ambos casos la mantisa se representa con signo-magnitud y el exponente en exceso a $2^{E-1} - 1$, siendo E el número de bits dedicados al exponente. El número de bits para cada formato es
 - Simple precisión: 1 bit de signo — 8 de exponente — 23 de mantisa (32 totales)
 - Doble precisión: 1 bit de signo — 11 de exponente — 52 de mantisa (64 totales)
- Como el primer dígito de una mantisa binaria normalizada siempre es 1 no hace falta incluirlo: solo se incluyen los dígitos de la parte fraccionaria.

Codificación de números reales

IEEE754

Ejemplo: El número X dado en formato de punto flotante según la norma IEEE 754 de simple precisión

$$X = \underbrace{0}_s \underbrace{10000011}_E \underbrace{111000000000000000000000}_M$$

- Con el valor de estos campos se calcula el valor en notación científica de acuerdo a la siguiente expresión:

$$(-1)^s \times 1.M \times 2^{E-127}$$

- Como el bit de signo es cero, el valor de la expresión es positiva
 $(-1)^s = (-1)^0 = +1$
- El valor del exponente es

$$10000011_2 - 01111111_2 = 00000100_2 = 4_{10}$$

- La Mantisa

$$1.M = 1.111000000000000000000000$$

$$= 1 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 1,875$$

El número final es

$$X = +1,875 \cdot 2^4 = 30_{10}$$

Codificación de números reales

IEEE754

Existen números especiales que se codifican siguiendo normas específicas:

- Cero: todo ceros (bit de signo 0 o 1).
- Números desnormalizados: exponente todos ceros, mantisa distinta de cero.
- Más o menos infinito: exponente todo unos, mantisa todo ceros (el bit de signo determina si es más o menos infinito).
- NaN (*not a number*): exponente todo unos, mantisa distinta de cero.

Codificación de números reales

IEEE754

Número no normalizado:

$$X = \underbrace{0}_s \underbrace{00000000}_E \underbrace{010000000000000000000000}_M$$

- Con el valor de estos campos se calcula el valor en notación científica de acuerdo a la siguiente expresión:

$$(-1)^s \times 0.M \times 2^{-126}$$

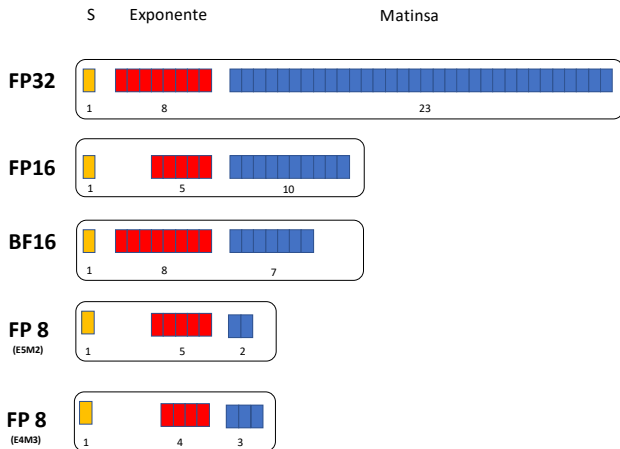
- Como el bit de signo es cero, el valor de la expresión es positiva
 $(-1)^s = (-1)^0 = +1$
- La Mantisa

$$0.M = 0.010000000000000000000000 = 1 \cdot 2^{-2}$$

El número final es

$$X = +1 \cdot 2^{-2} \cdot 2^{-127} = 1 \cdot 2^{-129}$$

Otras precisiones



Codificación de caracteres

Código ASCII

ASCII (*American Standard Code for Information Interchange*) es un esquema binario de codificación de caracteres basado en el orden del alfabeto en inglés.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Codificación de caracteres

ISO-8859

- **ISO-8859:** es un esquema de codificación de caracteres. Utiliza 8 bits para codificar cada carácter.
- El esquema ASCII permitía la codificación de textos en inglés, mientras que el esquema ISO-8859 mejora el soporte para otros idiomas y se definen 15 partes. Por ejemplo: ISO-8859-1 permite la codificación de textos que usen el alfabeto latino.

Codificación de caracteres

UTF-8

UTF-8: esquema basado en símbolos de longitud variable (1-4 bytes por carácter). Los bits más significativos del primer byte indican la longitud de la secuencia.

- Con un byte: caracteres incluidos en US-ASCII. Primer bit siempre a 0. Por tanto, tenemos $2^7 = 128$ caracteres.
- Con dos bytes: Caracteres romances más signos diacríticos, y los alfabetos griego, cirílico, copto, armenio, hebreo, árabe, siríaco y Thaana. Primer byte empieza por 110 y el segundo por 10. Por tanto, tenemos, $2^{(16-3-2)} = 2^{11} = 2048$ caracteres.
- Con tres bytes: caracteres del grupo CJK: Chino, japonés y coreano. Primer byte empieza por 1110, segundo y tercero por 10. Tenemos $2^{(24-4-2-2)} = 65536$ caracteres.
- Con cuatro bytes: símbolos matemáticos y alfabetos clásicos para uso principalmente académico. Primer byte empieza por 11110, segundo, tercer y cuarto byte por 10. Por tanto, tenemos $2^{(32-5-2-2-2)} = 2^{21} = 2097152$ caracteres.